Assistant vocal local pour Jeedom (sans Home Assistant)

Discussion avec Claude Code, extraite le 09/11/2026 à 09:17 (heure de Paris). Seuls les messages échangés figurent ici, sans les appels d'outils.
Les fichiers produits ont depuis été déplacés dans `C:\Travail\Acorel\Git\test-audio-jb` : les liens vers le dossier temporaire cités dans la discussion ne sont plus valides.

Benjamin 09/11/2026 08:55

Voici ce que je veux réaliser :

- Avoir un satellite audio sur un rpi3 avec un pihat seed 2mic qui écoute l'audio et le transmet à un container lxc sur une autre machine
- Sur le container lxc pouvoir réagir à un wakeword, déclencher un STT et streamer le message à ma domotique jeedom via le moteur d'interraction
- Le moteur STT doit être en local (whisper?) et rien ne doit être analysé hors de mon système

Voici ce qui est en place :

- J'ai un rpi3 sous debian 12 avec le pihat installé et les drivers configurés. Il est bien reconnu sur alsamixer
- j'ai un container lxc avec whisper d'installé sous environnement python et un wyoming server

Je ne veux pas :

- Dépendance à homeassistant
- analyse audio hors système local

Donne moi les procédures et le code source pour faire cette installation et si l'architecture n'est pas optimale, propose moi une solution alternative en respectant les contraintes ci-dessus

Claude 09/11/2026 08:58

Je vérifie le protocole Wyoming directement dans les sources (satellite, wake word, Whisper) et l'API Jeedom avant d'écrire le code du pont.

Sources de Wyoming lues (satellite, openWakeWord) ; je termine avec Whisper et Jeedom, puis j'écris le pont.

Le protocole et l'API Jeedom sont vérifiés dans les sources ; derniers contrôles sur Piper et les noms de modèles Whisper, puis j'écris le pont.

Je vérifie trois derniers détails (modes d'accès API Jeedom, modèles personnalisés openWakeWord, API serveur Wyoming) avant d'écrire.

Le pont et le faux satellite sont écrits ; je les teste maintenant contre des services simulés (wake word, Whisper, Piper, Jeedom).

Now generating a spoken WAV with Windows speech synthesis to feed the VAD real speech.

Les trois scénarios passent (wake word local, wake word distant, silence → erreur propre). Petit correctif d'espaces dans la réponse Jeedom, puis j'écris les unités systemd et la procédure.

Writing the systemd units for the Pi and the container.

Units écrites ; je rédige maintenant la procédure complète (README) avec l'architecture conseillée.

Final re-run of the end-to-end tests on the finished code, then packaging the archive.

Je t'ai préparé une archive avec la procédure complète et le code. Le pont tourne de bout en bout contre des services simulés, mais **rien n'a encore été essayé sur ton vrai Pi, ton Whisper ni ton Jeedom**.

La pièce qui manque

Dans ton installation actuelle, rien ne relie le satellite à Whisper, ni Whisper à Jeedom :

- `wyoming-satellite` **est un serveur.** Il attend qu'un « chef d'orchestre » se connecte à lui pour lui demander de streamer. D'habitude, ce rôle est tenu par Home Assistant.
- **Ton serveur Wyoming Whisper ne fait que transcrire** l'audio qu'on lui envoie.

J'ai donc écrit `voice_bridge.py`, qui tourne sur le container et prend la place de Home Assistant :

1. il se connecte à chaque satellite ;
2. il détecte le wake word, avec openWakeWord en local ;
3. il envoie la commande à Whisper et repère la fin de la phrase ;
4. il transmet le texte au moteur d'interaction Jeedom (`interact::tryToReply` en JSON-RPC, la clé API part dans le corps de la requête et non dans l'URL) ;
5. en option, Piper lit la réponse de Jeedom sur le haut-parleur du Pi.

Il gère plusieurs satellites, se reconnecte tout seul et maintient la connexion par ping/pong. J'ai vérifié le protocole et l'API Jeedom directement dans leurs sources.

Ton architecture n'est pas optimale : je recommande le wake word sur le Pi

Avec le wake word sur le container, **le micro est streamé en continu sur le réseau, 24 h/24**. Or Wyoming n'a ni authentification ni chiffrement : n'importe quelle machine du réseau local qui se connecte au port 10700 entend la pièce. Ça respecte tes contraintes, mais c'est le point faible.

Je te conseille plutôt de faire détecter le wake word par le Pi, avec openWakeWord en local :

- **Rien ne sort du Pi avant le wake word.**
- **Le Pi 3 a la puissance nécessaire :** il est de la même classe que le Pi Zero 2 W, pour lequel la détection locale est prévue.
- **Le container ne fait plus que Whisper, Jeedom et Piper.**

Passer d'un mode à l'autre ne change qu'une ligne du service satellite : le pont s'adapte tout seul. Il faut un OS 64 bits sur le Pi (`uname -m` doit afficher `aarch64`).

Procédure (détaillée dans le README)

1. **Sur le Pi :**
   - installer `wyoming-satellite` 1.4.1 avec réduction de bruit et contrôle de gain ;
   - créer le service systemd (le périphérique ALSA est à vérifier avec `arecord -L`) ;
   - en option, ajouter openWakeWord en local et les LED du HAT ;
   - mettre en place le **pare-feu nftables, indispensable** : seul le container doit pouvoir joindre le port 10700.
2. **Sur le container :**
   - relier ton Whisper (`--language fr`, modèle `small`) ;
   - installer openWakeWord, seulement si tu gardes ton architecture ;
   - installer Piper si tu veux la réponse vocale ;
   - installer le pont, remplir `/etc/voice-bridge/config.toml`, puis lancer `--check` (teste chaque service et la clé API Jeedom sans rien déclencher) et `--ask "…"` (envoie une phrase à Jeedom).
3. **Dans Jeedom :** récupérer la clé API, vérifier que l'API JSON-RPC est accessible (idéalement réservée à l'IP du container), puis créer tes interactions.

`tools/fake_satellite.py` rejoue un WAV comme un satellite : tu peux tester toute la chaîne du container avant de brancher le Pi.

Tests effectués

J'ai fait trois scénarios avec une vraie voix de synthèse, et les trois se comportent comme prévu :

- **Wake word sur le Pi :** la commande est transcrite, envoyée à Jeedom, et la réponse est lue.
- **Wake word sur le container :** même résultat.
- **Silence après le wake word :** le pont renvoie une erreur propre au satellite, qui repasse en attente.

Whisper, openWakeWord, Piper et Jeedom étaient simulés.

À savoir

- `wyoming-satellite` **n'est plus maintenu.** Son remplaçant utilise le protocole ESPHome et vise Home Assistant. Je l'ai figé en version 1.4.1, qui fonctionne ; le pont ne dépend que du protocole Wyoming, donc tu pourras changer de satellite plus tard.
- **Les wake words fournis sont en anglais** (`okay_nabu`, `hey_jarvis`…). Pour un wake word en français, il faut entraîner un modèle openWakeWord et déposer le `.tflite` dans `custom/`.
- **Whisper invente parfois des phrases dans le silence** (« Sous-titres réalisés par la communauté d'Amara.org »…). Le pont les filtre avant Jeedom, et la liste est modifiable dans la config.

Fichiers :

- jeedom-voice.tar.gz : l'archive à copier sur le Pi et le container
- README.md : la procédure complète
- voice_bridge.py : le code du pont

Ces fichiers sont dans un dossier temporaire de la session : télécharge l'archive pour la garder.

Benjamin 09/11/2026 09:14

Déplace les résultats dans le répertoire test-audio-jb et extrait la discussion que nous avons eu en PDF